

PMS Multitenant — QA Automation & Security Report

Cross-tenant vulnerabilities, authentication gaps, and the QA framework built to catch them

Prepared: 2026-09-12 · Environment tested: **staging** (production untouched)
Systems: pms-frontend-multitenant (admin), pms-client-multitenant (client), 15 backend microservices
Method: read-only source inspection + live black-box testing with two isolated QA tenants

Executive summary

An independent QA automation suite (Playwright + TypeScript) was built to test the PMS platform and report defects. Run against staging, it confirmed **eight application issues**, four of them security-critical, plus several supporting observations.

The most serious is a **multi-tenant isolation failure** in `account-contact-backend` : any authenticated admin could read and modify *another tenant's* accounts and contacts, and — chained with a credential-exposure bug and a public password-set endpoint — take over another tenant's client accounts entirely. The root cause is a single, easily-overlooked coding idiom repeated across one service. This vulnerability has been **fixed in the local codebase** (not yet deployed) with regression tests added.

#	Severity	Component	Issue	Status
1	P0	account-contact-backend	Cross-tenant read of accounts/contacts	Fixed (local)
2	P0	account-contact-backend	Cross-tenant write (7 handlers)	Fixed (local)
3	P0	account-contact-backend	Password hashes & activation tokens in API responses	Open
4	P1	client-sidebar-backend	No authentication on any route	Open
5	P1	sidebar-backend	<code>/api/newsidebar</code> routes unauthenticated	Open
6	P1	folder-mangement	<code>/tempfolder/docManagement</code> router unauthenticated	Open
7	P2	jobs-backend	<code>/jobs/count</code> & <code>/jobs/list</code> always 500	Open
8	P3	nginx (staging)	Bare <code>/admin</code> , <code>/client</code> serve default page	Open

Severity: **P0** critical (security / data loss) · **P1** major · **P2** broken with workaround · **P3** cosmetic / config.

The bugs, in detail

P0 #1 — Cross-tenant READ of accounts and contacts

Component account-contact-backend

Endpoints GET /api/clientaccounts/:id , GET /api/contacts/contact/:id

Status **Fixed in local working copy (undeployed)**

What it is

The platform is multi-tenant: every account, contact and record carries a `tenantId`, and a user should only ever see data belonging to their own tenant. This endpoint ignored that boundary — an admin logged into *Tenant A* could fetch *Tenant B*'s account or contact simply by knowing (or guessing) its ID.

Root cause

The handler tried to scope the query to the tenant, but did so incorrectly:

```
// BROKEN
const account = await Account.findById({ _id: accountId, tenantId: req.user.tenantId });
```

In Mongoose, `findById(x)` is shorthand for `findOne({ _id: x })`. When `x` is an *object*, Mongoose casts it to an `ObjectId` by reading its `_id` property and **silently discards every other key** — so `tenantId` was thrown away and the query became "find by id, any tenant." It *looks* tenant-scoped on the screen, which is exactly why it survived code review for months.

```
// FIXED
const account = await Account.findOne({ _id: accountId, tenantId: req.user.tenantId });
```

The proof it was a mistake, not a design choice: the *list* endpoints in the same service use `find({ tenantId })` correctly and are properly isolated.

Impact

Confidentiality breach across tenants — names, contact details, company data, account structure of unrelated customers. Also the first link in the account-takeover chain (see #3).

How it was detected

Spotted by source inspection, then proven live: as Tenant A, GET of Tenant B's account and contact IDs both returned **HTTP 200 with the other tenant's data**, in both directions.

P0 #2 — Cross-tenant WRITE (modify / attach / detach / archive)

Component	account-contact-backend
Handlers	updateAccount, updateContactwithoutPassword, uploadProfilePicture, toggleContactLogin, addContactsToAccount, removeContactFromAccount, updateAccountActiveStatus
Status	Fixed in local working copy (undeployed)

What it is

The same missing boundary, but on *write* operations: an admin of one tenant could edit another tenant's account fields, change a contact's permissions, attach or remove contacts, upload a profile picture, or bulk-archive accounts that don't belong to them.

Root cause

Two variants of the same class of mistake:

(a) **Object passed to `findByIdAndUpdate`** — identical casting flaw as #1, but on an update:

```
// BROKEN – tenantId dropped; writes to any tenant's record
await Account.findByIdAndUpdate({ _id: req.params.id, tenantId: req.user.tenantId },
                              { $set: req.body });
// FIXED – tenantId enforced in filter; _id/tenantId stripped from body
const { _id, tenantId, ...updates } = req.body;
await Account.findOneAndUpdate({ _id: req.params.id, tenantId: req.user.tenantId },
                              { $set: updates });
```

(b) **tenantId in the wrong position** in a bulk update:

```
// BROKEN – filter matches ANY tenant; the stray tenantId key
// would be folded into the update, reassigning matched accounts
await Account.updateMany({ _id: { $in: ids } },
                        { $set: { active }, tenantId: req.user.tenantId });
// FIXED – tenantId is part of the filter
await Account.updateMany({ _id: { $in: ids }, tenantId: req.user.tenantId },
                        { $set: { active } });
```

Impact

Integrity breach: tampering with another tenant's data. The bulk-archive variant additionally risked **reassigning** another tenant's accounts into the attacker's tenant (the stray `tenantId` key). And `updateAccount`'s `$set: req.body` was an unfiltered mass-assignment — a caller could set fields they should never control.

How it was detected & confirmed

Found by auditing every write handler in the service. Confirmed by **one controlled, reversible probe**: as Tenant A, a single benign field (`clientType`) on Tenant B's account was changed → HTTP 200 → verified as Tenant B that it changed → restored to the original value. No data left altered; only disposable QA tenants used.

P0 #3 — Password hashes and activation tokens exposed in API responses

Component	account-contact-backend
Endpoint	GET /api/contacts/contact/:id
Status	Open

What it is

The contact-detail response includes the contact's **bcrypt password hash** and, for any not-yet-activated contact, the raw **activation token** — neither of which should ever leave the server.

Root cause

`getContactById` returns the raw Mongoose document with no field projection, and the schema declares `password` and `activationToken` as ordinary fields (no `select:false`, no `toJSON` transform to strip them). So they serialize straight into JSON.

```
// Recommended fix (schema)
password:      { type: String, select: false },
activationToken: { type: String, select: false },
// plus a toJSON transform that deletes them, as defense in depth
```

Impact

Password hashes are offline-crackable. Activation tokens are live credentials — anyone holding one can set that account's password. This is the second link in the takeover chain below.

How it was detected

Observed directly in the JSON body when reading a contact. (A first version of the QA test missed it — see the "false negative" note under QA-framework bugs.)

The account-takeover chain. Bugs #1 + #3 combine with a *public* (unauthenticated) `POST /api/contacts/activate/set-password/:token` endpoint into full takeover of another tenant's client accounts:

1. Read a foreign tenant's contact by ID (**#1**).
2. Extract its activation token from the response (**#3**).
3. Set a password of your choosing via the public endpoint.
4. Log in to the victim tenant's client portal as that contact.

Bug #2's `toggleContactLogin` even mints a *fresh* token on demand, so already-activated contacts are reachable too. Each link was verified individually; the end-to-end takeover was deliberately **not** executed across tenants. Fixing the tenant scoping (**#1/#2**) breaks the chain at step 1; fixing **#3** breaks it at step 2 — both should be done.

P1 #4 — client-sidebar-backend has no authentication at all

Endpoints GET/POST/PATCH/DELETE /clientsidebar/...

Status **Open**

What it is & root cause

Every route in this service responds to unauthenticated requests. The service has no auth middleware anywhere — no JWT verification, no `protect`. Confirmed live: `GET /clientsidebar/` returns 200 with a JSON body and no token supplied.

Impact

Anyone on the internet can read and modify sidebar/navigation data. Control comparison: other services correctly return 401 unauthenticated on the same host, so this is a per-service gap, not a global misconfiguration.

P1 #5 — sidebar-backend /api/newsidebar routes unauthenticated

Status **Open**

What it is & root cause

This service *does* have the `protect` middleware and applies it to one route (`GET /api/sidebar/`), but the entire `/api/newsidebar` router — including write routes — has no `protect` applied. An unauthenticated write reaches the database layer (observed as a schema-validation 400 rather than a 401, proving it passed the auth layer that should have stopped it).

Impact

Unauthenticated read and write of sidebar configuration affecting the app's navigation for all users.

P1 #6 — folder-mangement /tempfolder/docManagement router unauthenticated

Status **Open**

What it is & root cause

This service is otherwise protected — four of its five routers apply `protect` — but the `docManagement` router was mounted without it. Confirmed live: `GET /tempfolder/docManagement/list` returns 200 with no token, while `GET /accounts/docs/list` on the *same service* correctly returns 401.

Impact

Unauthenticated access to document-management operations (list, and the router's write routes: folder/file upload, delete, move, rename). Easy to miss precisely because the rest of the service is protected.

P2 #7 — jobs-backend: `/jobs/count` and `/jobs/list` always return 500

Status

Open

What it is & root cause

Classic Express **route shadowing**. The parameterised route is registered before the literal ones:

```
router.route("/jobs/:id")      // registered first – matches "/jobs/count"
router.get("/jobs/count", ...) // never reached
router.get("/jobs/list", ...)  // never reached
```

A request to `/jobs/count` is captured by `/jobs/:id` with `id = "count"`, and Mongoose then throws casting `"count"` to an ObjectId → HTTP 500. The 3-segment siblings (`/jobs/count/active`) are unaffected and work, which confirms the diagnosis.

Fix

Declare the literal routes before `/jobs/:id`. (The proposal service already does exactly this, with a comment — the convention exists, it just wasn't applied here.)

P3 #8 — Bare `/admin` and `/client` serve the nginx default page

Status

Open (nginx config, not app code)

What it is & root cause

The apps are only reachable with a trailing slash (`/admin/` , `/client/`). The bare paths and the site root return nginx's stock "Welcome to nginx!" page. Consistent with the SPA locations being defined with a trailing slash while a catch-all serves the default root.

Impact

Low, but a user typing or sharing the bare URL sees a broken-looking page. It also masked a QA-side navigation bug (see below).

Related findings (classified, not "bugs to fix")

- **Contact activation emails fail silently** — the API returns 201 but no email is sent. Root cause: tenant-scoped email requires the tenant admin's connected Gmail, which new tenants don't have. **Reclassified as EXPECTED BEHAVIOR** — the platform intentionally keeps notifications off except for selected accounts. Side effect: invoice and proposal creation hard-fail 400 "Please connect Gmail first", which blocks two critical-flow tests.
- **Public activate/set-password/:token** — unauthenticated by necessity, but it is the final link in the takeover chain; worth reviewing once #1/#3 are fixed.
- **Client app logout** navigates to /client/login, a route that doesn't exist.

The fix that was applied (tenancy, #1 & #2)

At explicit direction, the tenancy vulnerability was fixed in the **local working copy** of `account-contact-backend` — uncommitted and undeployed (the running services live on a remote host, so no environment was altered):

- **10** `findById({_id, tenantId})` reads → `findOne({_id, tenantId})`
- **3** object-arg `findByIdAndUpdate({...})` writes → `findOneAndUpdate({_id, tenantId})`, with `_id` / `tenantId` stripped from the mass-assigned body
- **1** `updateMany` with `tenantId` misplaced → moved into the filter

Verified: no residual object-arg calls, both files pass `node --check`, and the two legitimate string-argument `findById()` calls were left untouched. A sweep confirmed the idiom exists in **no other service** — the fix is contained. Regression tests were added for cross-tenant reads and all affected write paths; the write tests capture-attempt-restore so they are safe to run even against a still-vulnerable target, and are gated behind `ALLOW_TENANCY_WRITE_PROBES` to run after deployment.

Bugs found in the QA framework itself

Running the suite against the real system exposed defects in the test code. Recorded because each silently broke a test before it was understood:

QA bug	Effect	Fix
Assumed <code>{data:[...]}</code> list envelope	Real endpoints return bare arrays / <code>{jobList}</code> / <code>{pipeline}</code> ; parsing got <code>undefined</code> and a tenancy test skipped past a confirmed P0	<code>extractList()</code> normalizer
Field-exposure check read top-level keys only	Record wrapped in <code>{success, data:{...}}</code> ; the hash sat one level down → false negative	<code>hasKeyDeep()</code> recursive check
<code>page.goto('/login')</code>	Leading slash resolves against the origin, discarding the <code>/admin</code> base path → hit nginx's default page	Navigate via page objects (absolute URLs)
Admin login filled only email + password	Form also requires a duration select and a terms checkbox → login never submitted	Page object sets all four inputs
Paths without trailing slash	nginx 301; a followed 301 on POST drops the body	<code>maxRedirects:0</code> so it fails loudly
Timestamped report filenames / truncated slugs	Duplicate files each run; distinct failures collapsed into one file	Stable names; slug from tail + hash

Current suite status (staging, fix not yet deployed)

56 passed · 6 failed · 13 skipped.

- **6 failures** — the 2 cross-tenant reads (fixed locally, pass once deployed), the field exposure (#3), and the 3 auth-boundary gaps (#4–#6). All real and still live on staging.
- **13 skipped** — 7 gated write probes (run post-deploy), 2 jobs-tenancy (no jobs in Tenant B yet), 4 critical flows (blocked as documented).
- Both frontends log in and render end-to-end; smoke and auth-boundary pass for the 12 services that enforce auth correctly.

Recommended next steps

1. Deploy the `account-contact-backend` tenancy fix; re-run with `ALLOW_TENANCY_WRITE_PROBES=1` to confirm reads and all write probes go green.
2. Fix #3 (field exposure) — `select:false` + `toJSON` transform. Small, contained, breaks the takeover chain independently.
3. Fix #4–#6 — add the missing `protect` middleware; the auth-boundary sweep confirms each.
4. Fix #7 (route order) and #8 (nginx trailing slash).
5. Seed a pipeline + job in Tenant A to unblock critical flow 1 and the jobs-tenancy tests.
6. Extend tenancy/functional coverage to the 8 services with only smoke coverage.

Guarantees held throughout. No production system was tested or modified. No application source was changed except the one directed tenancy fix, which remains local and uncommitted. No commits were made in any repository. All test data was created via supported APIs in dedicated QA tenants; no credentials or tokens appear in this report.

PMS QA Automation — generated from the project's bug-reports, ARCHITECTURE.md and CHARTER.md. Per-bug reproduction steps and raw evidence are in the `bug-reports/` directory of the pms-qa-automation repository.